Task 2

Michael König <m.koenig@kit.edu>
Paul Seehofer <paul.seehofer@kit.edu>

Deadline for submission: 2026-05-13T15:45

In this task, we want to develop a more sophisticated routing application. To evaluate correct routing behavior we will program our own monitoring application.

Task 2.1: Monitoring

- **Filename**: ppsdn26_uxxxx_task21.py
- **Scenario file**: `routing.yaml`
- **Automatic evaluation**: PARTIAL
- **Topology**: See Figure 1 (8 switches, 4 traffic generating hosts)
- **Traffic**: Each one of the hosts H1-H4 will send arbitrary IP flows to the other three hosts via the switches S1-S8. The source IP address of a flow belongs to the IP range of the sending host (i.e., 10.0.0.0/8 for H1), while the destination IP lies within the address range of any one of the other three hosts.
- **Assignment**: Program a set of predetermined flow rules that route traffic to the correct destination hosts based on IP destination addresses. You may choose these flow rules freely, as long as they deliver all traffic to the correct hosts. Furthermore, use the ryu switching hub to concurrently poll the statistics of the flow tables of each of the 8 switches S1-S8. Use this information to have your app (textually) visualize the flow of packets through the network over time.
- **Hints/Conditions**
  - Use the correct port numbers depicted in Figure 1
  - Repeatedly query statistics from the switches using OpenFlow protocol messages.
  - Use the ryu switching hub to handle concurrent operations and asynchronous messages delivered to your application (i.e., the statistics).
  - The automatic evaluation will only verify correct forwarding behavior, not your visualization of the network traffic.
  - You may want to use the python directive `print('$SDNC_CLEAR_SCREEN$')` to clear the output of the controller window before printing flow statistics.

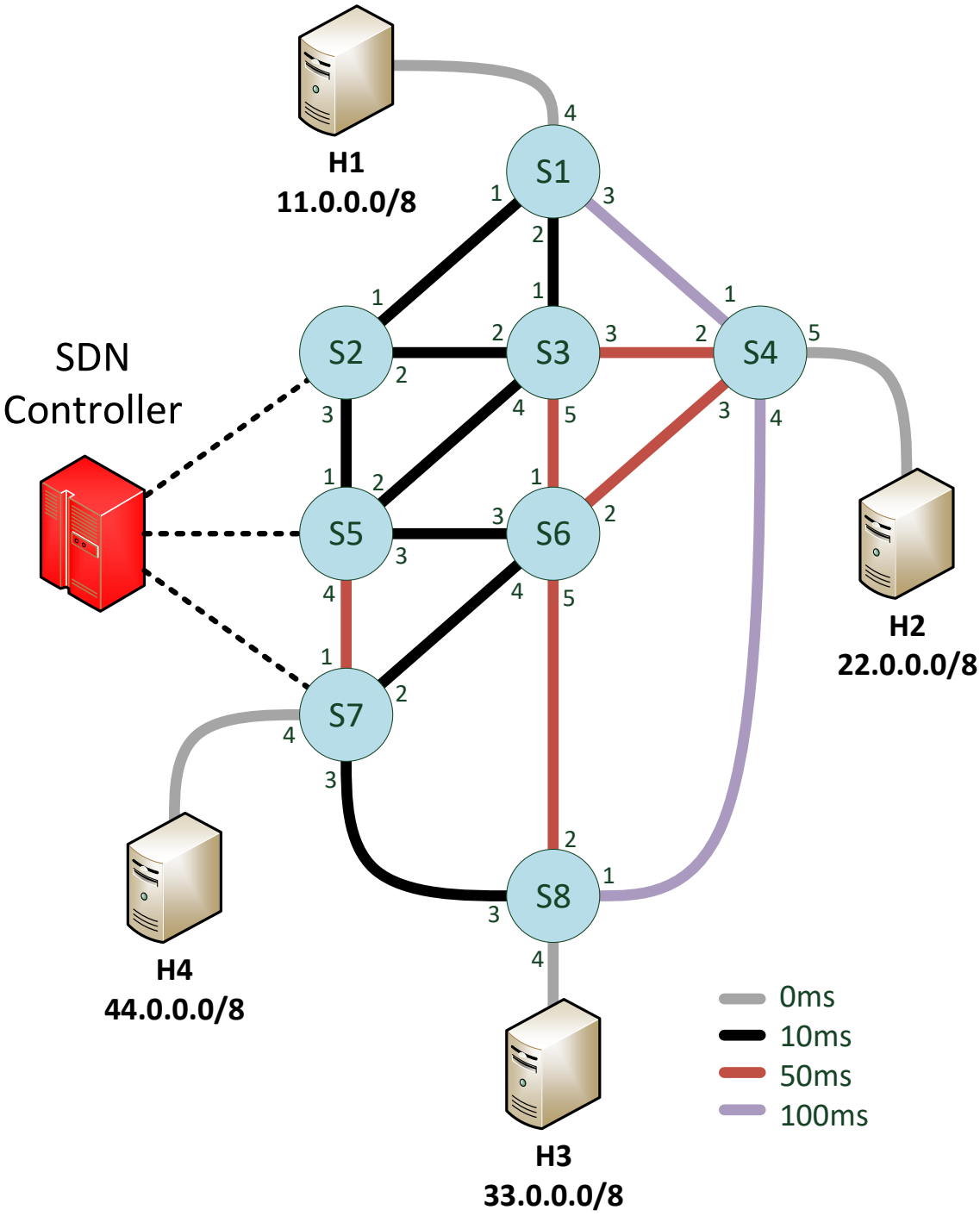

Figure 1: Scenario Task 2

Task 2.2: Shortest-Path Routing

- **Filename**: ppsdn26_uxxxx_task22.py
- **Scenario file**: `routing_shortest_path.yaml`
- **Automatic evaluation**: YES
- **Topology**: See Figure 1 (8 switches, 4 traffic generating hosts)
- **Traffic**: Same setup as in task 2.1.
- **Assignment**: Instead of programming a predetermined set of flow rules, calculate shortest paths in the given topology (e.g., by using Dijkstra's algorithm). Program these flow rules into the switches and (visually) verify that traffic is routed correctly over the shortest paths using your monitoring implementation from task 2.1.
- **Hints/Conditions**
  - Use the correct port numbers depicted in Figure 1.
  - You do not need to perform automated topology discovery. Use your own (predefined) representation of the topology instead.
  - You may use graph libraries (please add references if you re-use existing code).
  - Visualizing the flows and number of processed packets for each flow on every switch is sufficient to evaluate the forwarding behavior.

Task 2.3: Latency-Based Routing

- **Filename**: ppsdn26_uxxxx_task23.py
- **Scenario file**: `routing_latency.yaml`
- **Automatic evaluation**: YES
- **Topology**: See Figure 1 (8 switches, 4 traffic generating hosts)
- **Traffic**: Same setup as in task 2.1.
- **Assignment**: Adjust your routing application from task 2.2 to respect the delay of each link as depicted in Figure 1. Route all traffic over the paths with lowest latency. (Visually) verify the correct routing behavior with your monitoring application from task 2.1.